Interactive Computational Complexity Learning Platform

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

K. Sai Satwik Reddy (2520030446)
K. Shiva Sai Krishna (2520030610)
K. Charan (2520030530)

Team Number: 21

Section: 01

Under the guidance of

Dr. CH Anuradha



Department of Computer Science and Engineering

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled Interactive Computational Complexity Learning Platform is a record of Bonafide work of K. Satwik-2520030446, K. Sai Krishna-2520030610, K. Charan-2520030530. submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project-based report entitled Interactive Computational Complexity Learning Platform is a Bonafide work done and submitted by K. Satwik – 2520030446, K. Sai Krishna – 2520030610, K. Charan - 2520030530. in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. CH Anuradha, Assistant Professor, Department of Computer Science and Engineering which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Venkateshwara Rao (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. Satwik - 2520030446

K. Sai Krishna – 2520030610

K. Charan - 2520030530

TABLE OF CONTENTS

S.No.	Contents
1.	Abstract
2.	Introduction
3.	Literature survey
4.	Problem Statement and Objectives
5.	Hardware and Software requirements
6.	Implementation
7.	Results & Discussion
8.	Conclusion & Future Scope
9.	References

1.ABSTRACT

The Interactive Computational Complexity Learning Platform is an educational web-based application designed to help students understand fundamental concepts of computational complexity and algorithm analysis through interactive visualizations, practical examples, and problem-solving activities. Computational complexity is an important area of computer science that focuses on analyzing the time and space required by algorithms to solve computational problems. However, concepts such as P, NP, NP-Complete, and NP-Hard can be difficult for students to understand through theoretical explanations alone. This project aims to simplify these concepts by providing an interactive and user-friendly learning environment that connects theoretical knowledge with practical demonstrations.

The platform focuses on explaining major complexity classes and demonstrating how different algorithms solve computational problems. It incorporates visual representations, algorithm demonstrations, and examples involving well-known problems such as the Travelling Salesman Problem (TSP), Knapsack Problem, Vertex Cover, and Hamiltonian Cycle. Through these examples, users can explore different problem-solving approaches and understand how algorithmic strategies influence performance and computational difficulty. The platform also introduces important techniques such as graph traversal, dynamic programming, backtracking, greedy algorithms, approximation algorithms, and polynomial-time reductions.

The system is designed to make computational complexity more accessible to students with different levels of programming and algorithmic knowledge. By presenting complex topics in a structured and visual manner, the platform supports self-paced learning and helps users develop a stronger understanding of algorithm efficiency, computational limitations, and the distinction between problems that can be solved efficiently and those for which efficient solutions are not generally known. It also encourages analytical thinking by allowing learners to compare approaches, examine their limitations, and understand the importance of selecting suitable algorithms for different computational problems.

Overall, the proposed platform combines educational content, algorithm visualization, interactive demonstrations, and assessment features into a single learning environment. It aims to bridge the gap between theoretical concepts and practical implementation, making the study of computational complexity more engaging, understandable, and effective. The project can serve as a supplementary learning tool for computer science students studying Data Structures and Algorithms, Design and Analysis of Algorithms, and related subjects. By emphasizing visualization and hands-on exploration, the platform seeks to strengthen students' problem-solving abilities and provide a practical foundation for understanding the complexity of computational problems.

2. INTRODUCTION

Computational complexity is an important area of computer science that focuses on analyzing the time and memory required by algorithms to solve computational problems. Understanding the efficiency of algorithms helps programmers select suitable approaches for solving problems effectively. However, concepts such as P, NP, NP-Complete, and NP-Hard can be difficult for students to understand through theoretical explanations alone.

The Interactive Computational Complexity Learning Platform is a web-based educational application designed to simplify these concepts through interactive visualizations, algorithm demonstrations, and practical examples. The platform provides a user-friendly environment where students can explore computational complexity, observe how algorithms work, and understand the differences between various problem-solving approaches.

The application covers important computational problems, including the Travelling Salesman Problem (TSP), Knapsack Problem, Vertex Cover, and Hamiltonian Cycle. It demonstrates techniques such as graph traversal, dynamic programming, backtracking, greedy algorithms, approximation algorithms, and polynomial-time reductions. Through step-by-step demonstrations and visual representations, users can better understand algorithm execution and the challenges involved in solving complex computational problems.

The platform also incorporates interactive quizzes, practice activities, and automated evaluation of problem instances to help users assess their understanding. These features encourage active learning and improve logical reasoning and problem-solving skills. The system is primarily intended for computer science students studying Data Structures and Algorithms and Design and Analysis of Algorithms.

Overall, the project aims to bridge the gap between theoretical knowledge and practical understanding by presenting complex concepts in a simple, visual, and engaging manner. The following sections of this documentation describe the project objectives, system requirements, design, implementation, major features, testing, results, and possible future enhancements.

3. LITERATURE SURVEY

3.1 Existing Systems

1. Algorithm Visualizer

Algorithm Visualizer is an interactive online platform that demonstrates algorithms through visual animations. It allows users to observe how algorithms execute and explore the logic behind their operations. The platform includes algorithm tutorials and code-based visualizations, helping learners connect programming with algorithm behavior. Our project follows a similar visual-learning approach while focusing more specifically on computational complexity concepts and problem classifications.

2. VisuAlgo

VisuAlgo is an educational platform that uses animations and interactive simulations to explain data structures and algorithms. It helps students understand operations such as graph traversal, sorting, and other algorithmic techniques through step-by-step visual representations. Its interactive design makes abstract algorithmic processes easier to follow. Our project builds on this approach by emphasizing P, NP, NP-Complete, NP-Hard, and the difficulty of computational problems.

3. MIT OpenCourseWare – Computational Complexity

MIT OpenCourseWare provides academic learning materials on algorithms and computational complexity. Its resources explain topics such as complexity classes, computational hardness, completeness, and reductions through lectures and notes. These materials offer a strong theoretical foundation for understanding computational complexity. Our project aims to complement such resources with a more interactive environment that combines explanations, visual demonstrations, and practice activities.

4. DSViz – Data Structures and Algorithms Visualizer

DSViz is a web-based educational application designed to demonstrate data structures and algorithms through step-by-step visualizations. It allows learners to observe algorithm execution and explore individual processing steps at their own pace. This approach supports practical understanding rather than relying only on written explanations. Our proposed platform applies similar visualization principles to computational complexity and selected problems such as Knapsack and Hamiltonian Cycle.

5. alg0.dev – Interactive Algorithm Visualizer

alg0.dev is an interactive learning tool that presents algorithms using animations, code highlighting, and step-by-step execution. It also displays variable changes and provides contextual explanations to help users understand how algorithms work. The platform demonstrates the value of combining source code with visual feedback. Our project extends this general learning approach by incorporating complexity classifications, problem-solving strategies, and assessment activities.

6. graphs4u – Graph Algorithm Visualizer

graphs4u is a browser-based tool that allows users to create graphs and run algorithms such as Breadth-First Search, Depth-First Search, Dijkstra's algorithm, and maximum flow. It provides practical demonstrations of how graph algorithms process vertices and edges. This is relevant to our project because several computational problems, including Vertex Cover and Hamiltonian Cycle, involve graph structures. Our platform aims to connect such demonstrations with explanations of computational complexity.

7. OpenDSA – Interactive Algorithm Learning Platform

OpenDSA is an open-source educational platform that provides interactive visualizations of data structures and algorithms. It is designed to support students, educators, and developers through visual learning resources and algorithm demonstrations. Its approach shows how interactive educational material can improve the understanding of programming concepts. Our project uses a similar educational approach while concentrating on computational complexity classes, reductions, and the analysis of difficult computational problems.

8. MIT Design and Analysis of Algorithms – Complexity and Approximation

MIT's Design and Analysis of Algorithms course provides academic resources on P, NP, NP-completeness, polynomial-time reductions, and approximation algorithms. These topics are directly relevant to understanding the theoretical foundations of our project. The course materials help explain why certain problems are computationally difficult and how approximation methods can be used. Our platform aims to make these topics more approachable through interactive demonstrations, examples, and quizzes.

3.2 Research Gap and Proposed System

The reviewed platforms provide valuable resources for learning algorithms, data structures, and computational complexity. However, they differ in their primary focus: some emphasize algorithm animations, while others concentrate on academic lectures or graph-based demonstrations. The Interactive Computational Complexity Learning Platform aims to bring relevant learning elements together in one

environment, with particular emphasis on P, NP, NP-Complete, NP-Hard, selected computational problems, algorithm demonstrations, and interactive assessments. The proposed system is intended to help students connect theoretical concepts with practical examples in a structured and accessible manner.

REFERENCES

Add the following URLs to the References section of your documentation.

1. Algorithm Visualizer. https://algorithm-visualizer.org/
2. VisuAlgo – Visualising Data Structures and Algorithms. https://visualgo.org/
3. MIT OpenCourseWare – Computational Complexity. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/resources/lecture-23-computational-complexity/
4. DSViz – Data Structures and Algorithms Visualizer. https://dsviz.app/
5. alg0.dev – Interactive Algorithm Visualizer. https://alg0.dev/
6. graphs4u – Graph Algorithm Visualizer. https://graphs4u.com/
7. OpenDSA – Interactive Algorithm Learning Project. https://github.com/soloshun/opendsa
8. MIT OpenCourseWare – Design and Analysis of Algorithms, Lecture Notes. https://live.ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/pages/lecture-notes/

4. Problem Statement and Objectives

1. Problem Statement

Understanding computational complexity is challenging for students due to the abstract nature of complexity classes and the lack of interactive, visual learning resources.

2. Problem Description

Students often struggle to understand P, NP, NP-Complete, and NP-Hard problems through theoretical explanations alone. An interactive platform is needed to demonstrate algorithms, visualize problem-solving techniques, and assess learners' understanding.

3. Objectives

- Simplify Learning: Explain computational complexity concepts through simple explanations and practical examples.
- Visualize Algorithms: Demonstrate algorithm execution using interactive visualizations and step-by-step examples.
- Explore Complexity Classes: Help students understand P, NP, NP-Complete, and NP-Hard classifications.
- Demonstrate Problem-Solving Techniques: Illustrate graph algorithms, dynamic programming, backtracking, greedy methods, and approximation algorithms.
- Improve Practical Understanding: Provide examples involving TSP, Knapsack, Vertex Cover, and Hamiltonian Cycle.
- Assess Learning: Include quizzes, practice activities, and automated evaluation to test users' understanding.
- Encourage Independent Learning: Provide a user-friendly platform for students to learn and practice computational complexity at their own pace.

5. HARDWARE AND SOFTWARE REQUIREMENTS

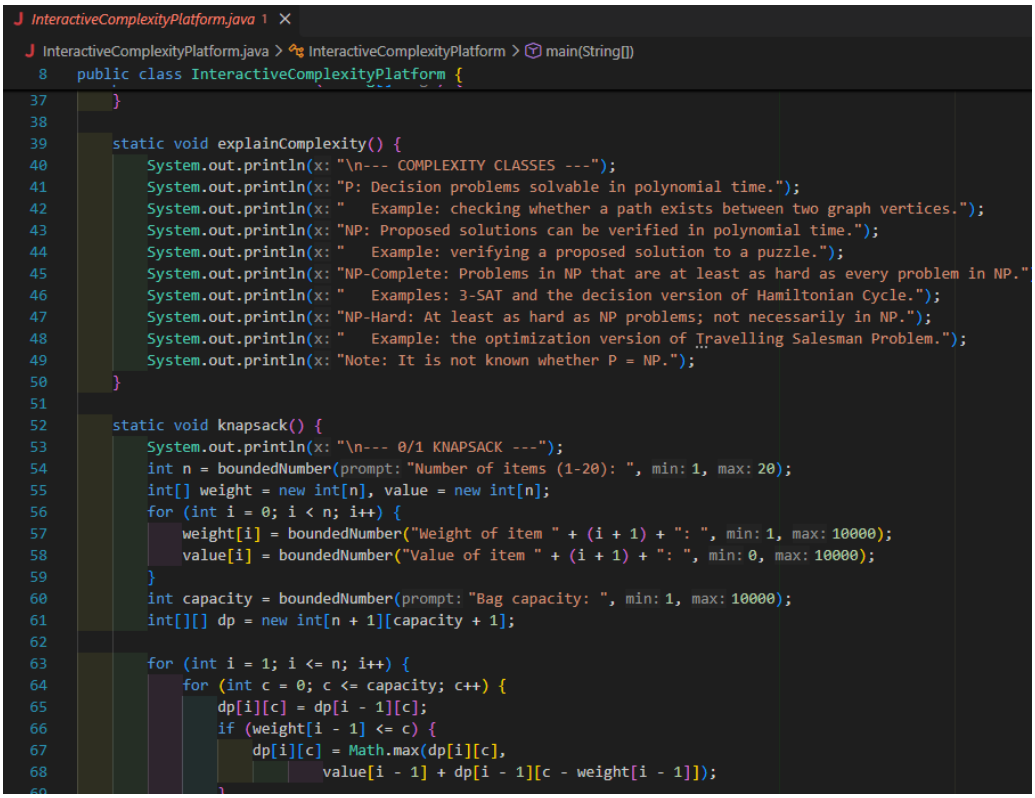
Hardware Requirements

- Processor: Intel Core i3 or equivalent and above.
- RAM: Minimum 4 GB (8 GB recommended).
- Storage: Minimum 500 MB of available disk space.

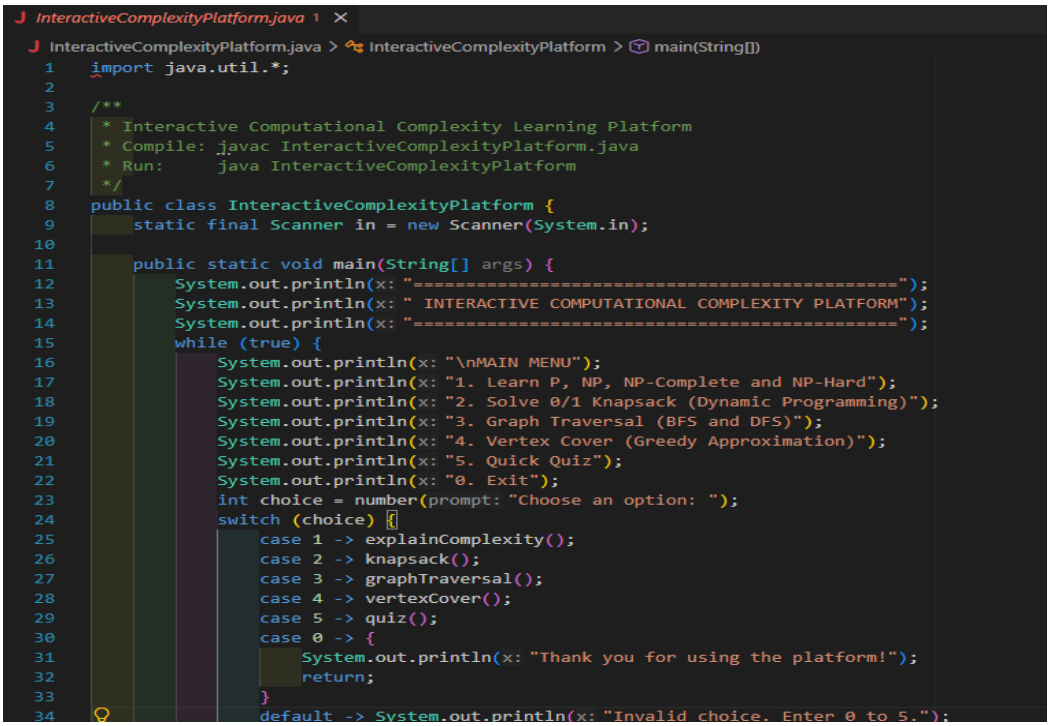
Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Frontend Technologies: HTML, CSS, and JavaScript.
- Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

6. IMPLEMENTATION



```
InteractiveComplexityPlatform.java 1
InteractiveComplexityPlatform.java > InteractiveComplexityPlatform > main(String[])
 8  public class InteractiveComplexityPlatform {
37      }
38
39      static void explainComplexity() {
40          System.out.println(x: "\n--- COMPLEXITY CLASSES ---");
41          System.out.println(x: "P: Decision problems solvable in polynomial time.");
42          System.out.println(x: "   Example: checking whether a path exists between two graph vertices.");
43          System.out.println(x: "NP: Proposed solutions can be verified in polynomial time.");
44          System.out.println(x: "   Example: verifying a proposed solution to a puzzle.");
45          System.out.println(x: "NP-Complete: Problems in NP that are at least as hard as every problem in NP.");
46          System.out.println(x: "   Examples: 3-SAT and the decision version of Hamiltonian Cycle.");
47          System.out.println(x: "NP-Hard: At least as hard as NP problems; not necessarily in NP.");
48          System.out.println(x: "   Example: the optimization version of Travelling Salesman Problem.");
49          System.out.println(x: "Note: It is not known whether P = NP.");
50      }
51
52      static void knapsack() {
53          System.out.println(x: "\n--- 0/1 KNAPSACK ---");
54          int n = boundedNumber(prompt: "Number of items (1-20): ", min: 1, max: 20);
55          int[] weight = new int[n], value = new int[n];
56          for (int i = 0; i < n; i++) {
57              weight[i] = boundedNumber("Weight of item " + (i + 1) + ": ", min: 1, max: 10000);
58              value[i] = boundedNumber("Value of item " + (i + 1) + ": ", min: 0, max: 10000);
59          }
60          int capacity = boundedNumber(prompt: "Bag capacity: ", min: 1, max: 10000);
61          int[][] dp = new int[n + 1][capacity + 1];
62
63          for (int i = 1; i <= n; i++) {
64              for (int c = 0; c <= capacity; c++) {
65                  dp[i][c] = dp[i - 1][c];
66                  if (weight[i - 1] <= c) {
67                      dp[i][c] = Math.max(dp[i][c],
68                              value[i - 1] + dp[i - 1][c - weight[i - 1]]);
```



```
InteractiveComplexityPlatform.java 1
InteractiveComplexityPlatform.java > InteractiveComplexityPlatform > main(String[])
 1  import java.util.*;
 2
 3  /**
 4   * Interactive Computational Complexity Learning Platform
 5   * Compile: javac InteractiveComplexityPlatform.java
 6   * Run:     java InteractiveComplexityPlatform
 7   */
 8  public class InteractiveComplexityPlatform {
 9      static final Scanner in = new Scanner(System.in);
10
11      public static void main(String[] args) {
12          System.out.println(x: "============================================");
13          System.out.println(x: " INTERACTIVE COMPUTATIONAL COMPLEXITY PLATFORM");
14          System.out.println(x: "============================================");
15          while (true) {
16              System.out.println(x: "\nMAIN MENU");
17              System.out.println(x: "1. Learn P, NP, NP-Complete and NP-Hard");
18              System.out.println(x: "2. Solve 0/1 Knapsack (Dynamic Programming)");
19              System.out.println(x: "3. Graph Traversal (BFS and DFS)");
20              System.out.println(x: "4. Vertex Cover (Greedy Approximation)");
21              System.out.println(x: "5. Quick Quiz");
22              System.out.println(x: "0. Exit");
23              int choice = number(prompt: "Choose an option: ");
24              switch (choice) {
25                  case 1 -> explainComplexity();
26                  case 2 -> knapsack();
27                  case 3 -> graphTraversal();
28                  case 4 -> vertexCover();
29                  case 5 -> quiz();
30                  case 0 -> {
31                      System.out.println(x: "Thank you for using the platform!");
32                      return;
33                  }
34                  default -> System.out.println(x: "Invalid choice. Enter 0 to 5.");
```

7. RESULTS & DISCUSSION

Output application run screen shots with their explanation in 2 lines.

```
==========================================
 INTERACTIVE COMPUTATIONAL COMPLEXITY PLATFORM
==========================================

MAIN MENU
1. Learn P, NP, NP-Complete and NP-Hard
2. Solve 0/1 Knapsack (Dynamic Programming)
3. Graph Traversal (BFS and DFS)
4. Vertex Cover (Greedy Approximation)
5. Quick Quiz
0. Exit
Choose an option: 1

--- COMPLEXITY CLASSES ---
P: Decision problems solvable in polynomial time.
   Example: checking whether a path exists between two graph vertices.
NP: Proposed solutions can be verified in polynomial time.
   Example: verifying a proposed solution to a puzzle.
NP-Complete: Problems in NP that are at least as hard as every problem in NP.
   Examples: 3-SAT and the decision version of Hamiltonian Cycle.
NP-Hard: At least as hard as NP problems; not necessarily in NP.
   Example: the optimization version of Travelling Salesman Problem.
Note: It is not known whether P = NP.
```

```
--- COMPLEXITY CLASSES ---
P: Decision problems solvable in polynomial time.
   Example: checking whether a path exists between two graph vertices.
NP: Proposed solutions can be verified in polynomial time.
   Example: verifying a proposed solution to a puzzle.
NP-Complete: Problems in NP that are at least as hard as every problem in NP.
   Examples: 3-SAT and the decision version of Hamiltonian Cycle.
NP-Hard: At least as hard as NP problems; not necessarily in NP.
   Example: the optimization version of Travelling Salesman Problem.
Note: It is not known whether P = NP.

MAIN MENU
1. Learn P, NP, NP-Complete and NP-Hard
2. Solve 0/1 Knapsack (Dynamic Programming)
3. Graph Traversal (BFS and DFS)
4. Vertex Cover (Greedy Approximation)
5. Quick Quiz
0. Exit
Choose an option: 2

--- 0/1 KNAPSACK ---
Number of items (1-20): 15
Weight of item 1: 2
Value of item 1: 2
Weight of item 2: 3
Value of item 2: 3
Weight of item 3: 3
Value of item 3:
```

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Interactive Computational Complexity Learning Platform aims to simplify the understanding of computational complexity through interactive visualizations, algorithm demonstrations, and practical examples. The project focuses on concepts such as P, NP, NP-Complete, and NP-Hard, along with algorithms and problems such as TSP, Knapsack, Vertex Cover, and Hamiltonian Cycle. By combining theoretical explanations with visual learning and assessment features, the platform provides a more engaging approach to learning complex algorithmic concepts. Overall, the project serves as a supplementary educational tool to improve students' understanding of algorithms, computational problem-solving, and complexity analysis.

Future Scope

The platform can be enhanced in the following ways:

1. Short-Term Enhancements

- Add more algorithm visualizations and interactive examples.
- Improve the user interface and navigation.
- Expand quizzes and practice questions.

2. Medium-Term Enhancements

- Integrate a code editor to allow users to write and execute algorithms.
- Add performance analysis to compare algorithm execution time and efficiency.
- Introduce user accounts to track learning progress and quiz scores.

3. Long-Term Enhancements

- Incorporate AI-based personalized learning recommendations.
- Support additional programming languages such as Java, Python, and C++.
- Develop advanced visualizations for complex algorithms and computational problems.

9. REFERENCES

- **Algorithm Visualizer.** https://algorithm-visualizer.org/
- **VisuAlgo – Visualising Data Structures and Algorithms.** https://visualgo.org/
- **MIT OpenCourseWare – Computational Complexity.** https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/resources/lecture-23-computational-complexity/
- **DSViz – Data Structures and Algorithms Visualizer.** https://dsviz.app/
- **alg0.dev – Interactive Algorithm Visualizer.** https://alg0.dev/
- **graphs4u – Graph Algorithm Visualizer.** https://graphs4u.com/
- **OpenDSA – Interactive Data Structures and Algorithms.** https://opendsa.org/
- **MIT OpenCourseWare – Design and Analysis of Algorithms.** https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/
- **GeeksforGeeks – P, NP, NP-Complete and NP-Hard Problems.** https://www.geeksforgeeks.org/dsa/introduction-to-np-completeness/
- **Stanford University – Algorithms Specialization and Learning Resources.** https://see.stanford.edu/